JAVA CHAPTER 9 — REFLECTION: INSPECTING CLASSES, METHODS, FIELDS & CONSTRUCTORS

Discover type information at runtime • Invoke dynamically • Use carefully

1 WHY IT MATTERS

- Used by frameworks, libraries and containers (e.g. dependency injection).
- Enables serialization, testing tools, ORMs and plugin systems.
- Lets you inspect types, methods, fields and annotations at runtime.
- Provides flexibility for frameworks that work with unknown types.

2 MENTAL MODEL

Class object	Inspect members	Read metadata / annotations	Create object or invoke	Framework behavior
Get a Class<?> from a type.	Find fields, methods, constructors, etc.	Get names, types modifiers and annotations.	Construct instances and call methods or access fields.	Use the information to provide dynamic features.

- Reflection lets you examine and use class information at runtime.
- It is often used by frameworks and libraries, not usually in everyday application code.

3 GETTING A CLASS OBJECT

`MyType.class`	Compile-time reference. Use when you know the type at compile time.
`obj.getClass()`	From an existing instance. Use when you have an object and want its runtime type.
`Class.forName("com.example.MyType")`	By fully qualified name. Use when the class name is known only at runtime (may throw ClassNotFoundException).

4 CORE APIS

API	Purpose
`Class<?>`	Represents a class or interface at runtime. Entry point for inspecting its members.
`Field`	Represents a field (variable). Read or modify values.
`Method`	Represents a method. Invoke it dynamically.
`Constructor<?>`	Represents a constructor. Create new instances.
`Modifier`	Utility to check modifiers (public, private, static, etc).
`Package`	Represents package information for a class.
`Annotation`	Read annotations present on classes, methods, fields, etc.

5 COMMON REFLECTION TASKS

- **Inspect method names** — Get all declared or public methods and their signatures.
- **List fields** — Get fields, their types, modifiers and current values.
- **Read annotations** — Check for annotations on classes, methods or fields.
- **Call a constructor** — Find a constructor and create a new instance.
- **Invoke a method** — Call a method with arguments and get the return value.
- **Read or set a field** — Get or update the value of a field (even private with access).

6 JAVA EXAMPLE

```java
import java.lang.reflect.*;
class Calculator {
    private int calculate(int a, int b) {
        return a + b;
    }
}
public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        // 1. Get Class object
        Class<?> clazz = Calculator.class;
        // 2. Get declared method with parameter types
        Method method = clazz.getDeclaredMethod(
            "calculate", int.class, int.class);
        // 3. Create instance using constructor
        Object obj = clazz.getDeclaredConstructor().newInstance();
        // 4. Allow access to private method and invoke it
        method.setAccessible(true);
        Object result = method.invoke(obj, 3, 4);
    }   System.out.println("Result: " + result);
}
```

Output (console)

```
Result: 7
```

7 ACCESS CONTROL & setAccessible

- Reflection can access private, protected and package-private members using `setAccessible(true)`.
- Useful for frameworks and testing tools.
- Should be used sparingly, as it breaks encapsulation.
- May be restricted in modern Java (module system, security manager, or runtime policies).

8 COMMON MISTAKES

- Using reflection when normal code would do.
- Ignoring checked exceptions from reflection APIs.
- Passing wrong parameter types to methods/constructors.
- Forgetting that reflection is slower and less type-safe.
- Breaking encapsulation or modifying private state.
- Not handling `NoSuchMethodException`, `IllegalAccessException` or `InvocationTargetException`.

9 PERFORMANCE & SAFETY

- Reflection is slower than direct method calls.
- May be restricted by the module system, security manager or runtime policies.
- Cache reflective lookups (Class, Method, Field) if used repeatedly.
- Validate input and handle exceptions properly.
- Use reflection with caution and only when needed.

10 QUICK RULES / AT A GLANCE

1. Prefer normal code first; use reflection only when needed.
2. Use reflection at boundaries (frameworks, libraries, plugins).
3. Cache Class, Method and Field objects for repeated use.
4. Handle all reflection exceptions and provide clear messages.
5. Respect encapsulation and avoid changing private state unless necessary.
6. Consider module restrictions and security policies.
7. Keep reflection code small, well-documented and test it.